Dashboard Admin - Visualizaciones Creativas

📊 Componentes de Visualización

1. **AnimatedKPICard** - Tarjetas KPI Animadas

Ubicación: `frontend/src/components/charts/AnimatedKPICard.jsx`

Características:

- ✨ Contador animado con spring physics
- 📈 Indicador de tendencia con comparación período anterior
- 🎨 Gradientes de fondo dinámicos
- 🔄 Animación de entrada escalonada
- 📊 Barra de progreso de tendencia

Uso:

```
<AnimatedKPICard
  title="Ingresos del mes"
  value={3200000}
  previousValue={2450000}
  icon={DollarSign}
  trend
  prefix="$"
  color="var(--color-success)"
  delay={0.1}
/>
```

Props:

- `title`: Título del KPI
- `value`: Valor actual (animado)
- `previousValue`: Valor anterior para comparación
- `icon`: Componente de icono (Lucide)
- `trend`: Mostrar barra de tendencia
- `prefix/suffix`: Formato del valor
- `color`: Color temático del card
- `loading`: Estado de carga
- `delay`: Delay de animación (escalonado)

---

2. **SparklineChart** - Mini Gráficos de Tendencia

Ubicación: `frontend/src/components/charts/SparklineChart.jsx`

Características:

- 📉 Gráfico de línea compacto e inline
- 🎨 Gradiente de relleno opcional
- ⚡ Animación suave de trazado
- 🔵 Punto pulsante en último valor
- 💫 Perfecto para mostrar en cards/tablas

Uso:

```
<SparklineChart
  data={[45, 52, 48, 61, 58, 72, 68]}
  width={80}
  height={30}
  color="var(--color-primary1)"
  fillColor="var(--color-primary1)"
  showDots={false}
/>
```

Props:

- `data`: Array de números o `{value: number}`
- `width/height`: Dimensiones del gráfico
- `color`: Color de la línea
- `fillColor`: Color de relleno con gradiente
- `strokeWidth`: Grosor de línea
- `showDots`: Mostrar puntos en cada valor
- `animate`: Activar animaciones

---

3. **ProgressRing** - Anillos de Progreso Circulares

Ubicación: `frontend/src/components/charts/ProgressRing.jsx`

Características:

- ⭕ Anillo circular animado
- 🎯 Perfecto para porcentajes y completitud
- 📊 Centro personalizable con valor/etiqueta
- 🌈 Animación de llenado suave
- 💅 Sombras y efectos visuales

Uso:

```
<ProgressRing
  progress={78.5}
  size={180}
  strokeWidth={14}
  color="var(--color-success)"
  label="Conversión"
  showPercentage
/>
```

Props:

- `progress`: Porcentaje (0-100)
- `size`: Tamaño del anillo en px
- `strokeWidth`: Grosor del anillo
- `color`: Color del progreso
- `backgroundColor`: Color del fondo
- `showPercentage`: Mostrar % en centro
- `label`: Etiqueta inferior
- `value`: Valor custom en lugar de %

---

4. **HeatMapChart** - Mapas de Calor

Ubicación: `frontend/src/components/charts/HeatMapChart.jsx`

Características:

- 🔥 Visualización de actividad por dimensiones
- 🎨 Escala de colores personalizable
- 📊 Perfecto para patrones día/hora
- 🖱️ Hover interactivo
- 🎭 Animación de entrada celda por celda

Uso:

```
<HeatMapChart
  data={[
    [{value: 45}, {value: 78}, {value: 23}],
    [{value: 91}, {value: 56}, {value: 67}]
  ]}
  xLabels={["9-12", "12-15", "15-18"]}
  yLabels={["Lun", "Mar"]}
  showValues
  cellSize={40}
/>
```

Props:

- `data`: Matriz 2D con `{value: number}`
- `xLabels`: Etiquetas eje X
- `yLabels`: Etiquetas eje Y
- `colorScale`: Array de colores (claro→oscuro)
- `showValues`: Mostrar números en celdas
- `cellSize`: Tamaño de celda en px
- `gap`: Espacio entre celdas

Casos de uso:

- Actividad por día/hora
- Rendimiento por categoría/mes
- Engagement por tipo/período

---

5. **ComparisonCard** - Tarjetas de Comparación

Ubicación: `frontend/src/components/charts/ComparisonCard.jsx`

Características:

- 📊 Comparación visual período actual vs anterior
- 📈 Barras de progreso proporcionales
- ⚡ Indicador de tendencia con %
- 🎨 Efecto shimmer en barra actual
- 💰 Diferencia destacada

Uso:

```
<ComparisonCard
  title="Ingresos Mensuales"
  currentValue={3200000}
  previousValue={2450000}
  formatter={(v) => formatCurrencyCLP(v)}
  color="var(--color-success)"
  delay={0.1}
/>
```

Props:

- `title`: Título de la comparación
- `currentLabel/previousLabel`: Etiquetas de períodos
- `currentValue/previousValue`: Valores numéricos
- `formatter`: Función de formato

- `color`: Color temático
- `delay`: Delay de animación

🎨 Sistema de Tabs del Dashboard

El nuevo dashboard está organizado en **5 tabs principales**, cada uno con visualizaciones específicas conectadas al backend:

Tab 1: Overview 📊

Vista general con métricas principales y resumen ejecutivo.

Visualizaciones:

- 4x `AnimatedKPICard` con métricas principales
- `AreaChart` para ingresos diarios
- `BarChart` para distribución de pedidos
- `BarChart` horizontal para top categorías
- `PieChart` para estado de inventario
- Grid de productos destacados con `SparklineChart`

Data backend:

- `analyticsApi.getDashboardMetrics()`
- `analyticsApi.getSalesAnalytics()`
- `analyticsApi.getTopProducts()`
- `analyticsApi.getCategoryAnalytics()`
- `analyticsApi.getStockAnalytics()`

Tab 2: Analytics 📈

Análisis profundo con comparativas y patrones.

Visualizaciones:

- 3x `ComparisonCard` para comparar períodos
- `ProgressRing` grande para conversión
- `HeatMapChart` para actividad por horario
- Métricas de visitantes vs compradores

Data backend:

- `analyticsApi.getConversionMetrics()`
- `analyticsApi.getSalesAnalytics()` (comparativas)
- Dashboard metrics para totales

Insights:

- Crecimiento período a período
- Tasa de conversión visual
- Patrones de comportamiento por día/hora

Tab 3: Products 📦

Análisis de productos e inventario.

Visualizaciones:

- 4x `AnimatedKPICard` con métricas de stock
- `BarChart` horizontal para top productos por ingresos
- `PieChart` para distribución de categorías

Data backend:

- `analyticsApi.getStockAnalytics()`
- `analyticsApi.getTopProducts({ limit: 8 })`
- `analyticsApi.getCategoryAnalytics()`

Métricas clave:

- Stock saludable / bajo / sin stock
- Performance por producto
- Distribución por categoría

Tab 4: Customers 👥

Comportamiento y segmentación de clientes.

Visualizaciones:

- 4x `AnimatedKPICard` con métricas de clientes
- `ProgressRing` para segmentación activos/inactivos
- `BarChart` dual para preferencias de categorías

Data backend:

- `analyticsApi.getConversionMetrics()`
- `analyticsApi.getCategoryAnalytics()`
- Dashboard metrics para totales

Insights:

- Tasa de conversión

- Segmentación de compradores
- Preferencias por categoría

Tab 5: Operations 🚚

Gestión operativa y órdenes.

Visualizaciones:

- 4x `AnimatedKPICard` con estados de órdenes
- Timeline de últimas órdenes con `StatusPill`
- Alertas de inventario animadas
- Lista de productos críticos

Data backend:

- `ordersApi.list({ limit: 4 })`
- `analyticsApi.getStockAnalytics()`
- Dashboard metrics para estados

Operaciones:

- Monitor de órdenes en tiempo real
- Alertas de stock crítico
- Estados de pedidos

🎭 Animaciones y Transiciones

Framer Motion

Todas las animaciones usan `framer-motion` para:

- ✨ Entrada/salida de componentes
- 🔄 Transiciones entre tabs
- 📊 Animación de gráficos
- ⚡ Hover effects
- 🎯 Spring physics para números

Patrones de Animación:

1. Entrada Escalonada (Stagger):

```
<AnimatedKPICard delay={index * 0.1} />
```

2. Transición entre Tabs:

```
<AnimatePresence mode="wait">
  <motion.div
    key={activeTab}
    initial={{ opacity: 0, x: -20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: 20 }}
  />
</AnimatePresence>
```

3. Hover Interactivo:

```
<motion.div whileHover={{ scale: 1.02, boxShadow: "var(--shadow-lg)" }} />
```

4. Contador Animado:

```
const spring = useSpring(0, {
  stiffness: 80,
  damping: 20,
  mass: 1,
});
```

🎨 Tokens de Diseño

Todos los componentes usan consistentemente los tokens de `tokens.css`:

Colores:

- `--color-primary1`: Principal
- `--color-success`: Métricas positivas
- `--color-warning`: Alertas
- `--color-error`: Crítico
- `--color-secondary1/2`: Categorías secundarias

Sombras:

- `--shadow-sm`: Cards normales
- `--shadow-md`: Hover
- `--shadow-lg`: Énfasis

Border Radius:

- `--radius-xl`: Contenedores principales (1rem)
- Rounded-2xl/3xl para cards

Transiciones:

- `--transition-base`: Transiciones estándar
- `duration: 0.5s` para animaciones principales

📱 Responsive

Todos los componentes son **completamente responsive**:

- Grid adaptable: `grid-cols-1 sm:grid-cols-2 lg:grid-cols-4`
- Tabs scrollables en móvil
- Gráficos con `ResponsiveContainer` de Recharts
- Breakpoints: `sm:` (640px), `lg:` (1024px)

🔄 Refetch y Sincronización

Actualización automática cada 5 minutos:

```
const DASHBOARD_STALE_TIME = 1000 * 60 * 5;
```

Botón de actualizar manual:

```
<Button onClick={refetch} disabled={isLoading}>
  <RefreshCw className={isLoading ? 'animate-spin' : ''} />
  Actualizar
</Button>
```

Carga paralela con `useQueries`:
Todas las requests se hacen simultáneamente para máxima velocidad.

🚀 Performance

Optimizaciones:

- ✅ `useMemo` para cálculos pesados
- ✅ `AnimatePresence` para transiciones suaves

- ✅ Lazy rendering de tabs
- ✅ Skeleton loaders durante carga
- ✅ Parallel data fetching

Bundle Size:

- Framer Motion: ~60KB (gzipped)
- Recharts: ~200KB (gzipped)
- Total nuevo: +260KB

📚 Ejemplos de Uso

Crear nuevo KPI:

```
<AnimatedKPICard
  title="Nueva métrica"
  value={1234}
  previousValue={980}
  icon={TrendingUp}
  trend
  color="var(--color-primary1)"
  loading={isLoading}
  delay={0.2}
/>
```

Agregar nuevo tab:

```
<TabsTrigger value="nuevo">
  <Icon className="mr-2 h-4 w-4" />
  Nuevo Tab
</TabsTrigger>

<TabsContent value="nuevo">
  {/* Contenido del tab */}
</TabsContent>
```

Heatmap personalizado:

```
<HeatMapChart
  data={miData}
  xLabels={["Ene", "Feb", "Mar"]}
  yLabels={["Producto A", "Producto B"]}
```

```
    colorScale={[
      "#FAF8F5", // Claro
      "#C9A88A",
      "#A67B5B",
      "#6B5444",
      "#52443A"  // Oscuro
    ]}
  />
```

🎯 Próximas Mejoras

1. **Filtros de fecha**: Selector de rango personalizado
2. **Modo comparación**: Ver múltiples períodos simultáneamente
3. **Export**: Descargar reportes en PDF/CSV por tab
4. **Real-time**: WebSocket para actualizaciones en vivo
5. **Drill-down**: Click en gráficos para ver detalle
6. **Favoritos**: Guardar configuración de tabs preferida

🐛 Troubleshooting

Los números no se animan:

- Verificar que `framer-motion` esté instalado
- Confirmar que `value` es numérico

Gráficos no cargan:

- Verificar data del backend en Network tab
- Confirmar estructura de datos coincide con props

Animaciones lentas:

- Reducir `delay` en componentes
- Usar `animate={false}` en modo dev

Tabs no cambian:

- Verificar estado `activeTab` en React DevTools
- Confirmar `onValueChange` está conectado

📖 Referencias

- Framer Motion Docs
- Recharts Docs
- Lucide Icons

- Tokens: `frontend/src/styles/tokens.css`